

2.1 Algorithms

Where this unit sits, what it covers, and how far

Where this unit sits

Qualification	GCSE Computer Science (9-1), specification code J277, from 2020
Component	Component 02: Computational thinking, algorithms and programming — 80 marks, 1 hour 30 minutes
What OCR says the component covers	computational thinking: algorithms, programming techniques, producing robust programs, computational logic and translators
This unit	topic 2.1 Algorithms
Checked	3 October 2026, at https://www.ocr.org.uk/qualifications/gcse/computer-science-j277-from-2020/specification-at-a-glance/

How far this has been checked. The topic numbers and titles below are the ones this course uses on its worksheets and slides. They were not re-checked against the published specification document when this map was built: the document could not be retrieved from this build environment. Check them against <https://www.ocr.org.uk/Images/558027-specification-gcse-computer-science-j277.pdf> before relying on this map for a scheme of work, and treat any finer subdivision as absent rather than assumed - no sub-point numbers are claimed anywhere in these documents, because none were verified.

Before it: 1.1 Systems architecture, 1.2 Memory and storage, 1.3 Computer networks, connections and protocols, 1.4 Network security, 1.5 Systems software, 1.6 Ethical, legal, cultural and environmental impacts of digital technology

After it: 2.2 Programming fundamentals, 2.3 Producing robust programs, 2.4 Boolean logic, 2.5 Programming languages and Integrated Development Environments, PY Python programming

The lesson sequence and the questions it answers

#	LESSON	THE BIG QUESTION	ACTIVITIES	MARKS
1	Abstraction studio a1-101	What is abstraction, and why is it needed?	13	30
2	Breakdown bay a1-102	What is decomposition, and how does a structure diagram help?	13	33
3	Algorithm workshop a1-103	What do we mean by thinking algorithmically?	13	36
4	Flowchart factory a1-104	How can an algorithm be described without ambiguity?	14	38
5	Pseudocode press a1-105	How is an algorithm written so that anyone can follow it exactly?	16	28
6	Search corridor a1-106	How does a linear search work, and when is it the right choice?	15	29
7	Halving hall a1-107	How does a binary search work, and why is it so much faster?	14	30

#	LESSON	THE BIG QUESTION	ACTIVITIES	MARKS
8	Bubble bay al-l08	How does a bubble sort work, and why is it slow?	15	23
9	Split and insert al-l09	How do merge sort and insertion sort work, and how do they compare with a bubble sort?	14	29
10	Fault finder al-l10	What kinds of error occur when programming, and how are they found?	14	30
11	Trace room al-l11	How and why do programmers use a trace table?	13	55
12	End of topic test al-l12-test	assesses the whole unit	0	0
13	Algorithm arena al-bonus	stretch work, no new content	1	0

Prior knowledge, vocabulary and skills

Assumed before the unit: the outcomes of 1.1, 1.2, 1.3, 1.4, 1.5, 1.6

Vocabulary introduced: Abstraction, Computational thinking, Decomposition, Structure diagram, Algorithm, Process, Flowchart, Decision, Pseudocode, Iteration, Linear search, Worst case, Binary search, Midpoint, Bubble sort, Pass, Merge sort, Insertion sort, Syntax error, Logic error, Trace table, Dry run

What a pupil does, not only reads: arena, bugline, code, match, mcq, multi, order, searchstep, sort, sortstep, trace

Coverage matrix

Every outcome this unit states, and whether it is taught, practised and assessed **within this unit**. Full means all three; partial means some; not covered means none of the three could be traced.

How this was worked out. The trace is derived by matching the content words of an outcome against the station text, the activities and the exam questions of the unit, not authored by hand. The words each match was made on are in build/coverage.json. It shows where to look; it does not certify that the alignment is right, and a full row has not been checked by a person either.

OUTCOME	LESSON	TAUGHT	PRACTISED	ASSESSED	COVERAGE
know what is meant by the term abstraction al-l01-o1	al-l01	yes	yes	yes	Full
know some examples of abstraction al-l01-o2	al-l01	yes	yes	yes	Full
understand why abstraction is needed in computational thinking al-l01-o3	al-l01	yes	yes	yes	Full
know what is meant by problem decomposition al-l02-o1	al-l02	yes	yes	yes	Full
know the advantages of decomposition when programming al-l02-o2	al-l02	yes	yes	yes	Full
know how to produce a structure diagram al-l02-o3	al-l02	yes	yes	yes	Full
understand how to solve problems by applying algorithmic thinking al-l03-o1	al-l03	yes	yes	yes	Full

OUTCOME	LESSON	TAUGHT	PRACTISED	ASSESSED	COVERAGE
be able to identify the inputs, processes and outputs of a problem al-l03-o2	al-l03	yes	yes	yes	Full
know the flowchart symbols al-l04-o1	al-l04	yes	yes	yes	Full
know how to make a flowchart al-l04-o2	al-l04	yes	yes	yes	Full
understand how to construct a program from a flowchart al-l04-o3	al-l04	yes	yes	yes	Full
know what is meant by the term pseudocode al-l05-o1	al-l05	yes	yes	yes	Full
understand how to write pseudocode al-l05-o2	al-l05	yes	yes	yes	Full
understand the OCR reference language al-l05-o3	al-l05	yes	yes	yes	Full
understand the linear search algorithm al-l06-o1	al-l06	yes	yes	yes	Full
understand that it is inefficient but simple and needs no ordering al-l06-o2	al-l06	yes	yes	yes	Full
understand the binary search algorithm al-l07-o1	al-l07	yes	yes	yes	Full
know the condition a list must meet for a binary search to work al-l07-o2	al-l07	yes	yes	yes	Full
understand which searching algorithm is quicker al-l07-o3	al-l07	yes	yes	yes	Full
understand the bubble sort algorithm al-l08-o1	al-l08	yes	yes	yes	Full
be able to show the steps of a bubble sort al-l08-o2	al-l08	yes	yes	yes	Full
know the advantages and disadvantages of a bubble sort al-l08-o3	al-l08	yes	yes	yes	Full
understand the merge sort algorithm al-l09-o1	al-l09	yes	yes	yes	Full
understand the insertion sort algorithm al-l09-o2	al-l09	yes	yes	yes	Full
be able to compare the three sorting algorithms al-l09-o3	al-l09	yes	yes	yes	Full
know what a syntax error is al-l10-o1	al-l10	yes	yes	yes	Full
know what a logic error is al-l10-o2	al-l10	yes	yes	yes	Full

OUTCOME	LESSON	TAUGHT	PRACTISED	ASSESSED	COVERAGE
be able to identify errors in code and suggest fixes a1-l10-o3	al-l10	yes	yes	yes	Full
know what a trace table is a1-l11-o1	al-l11	yes	yes	yes	Full
understand how and why trace tables help with debugging a1-l11-o2	al-l11	yes	yes	yes	Full
know how to complete a trace table a1-l11-o3	al-l11	yes	yes	yes	Full

31 full · 0 partial · 0 not covered, of 31 outcomes.

Gaps, assumptions and overlap

None found by the trace.

Assessed, but matching no outcome this unit states

Each of these is a real exam question on a worksheet in this unit. Either the outcome it tests is worded differently, or it is genuinely assessing something the unit does not claim to teach.

- a1-l104-x2 Explain why a decision symbol must have two arrows leaving it.
- a1-l105-x2 Explain the difference between a for loop and a while loop.
- a1-l110-x2 A program runs but always outputs one more than it should. What type of error is this? Explain your answer.

What could not be traced

The wall text of a1 - bonus is painted into the 360 image and held nowhere as text, so an outcome in that lesson can only be traced through its activities and its exam questions. Read the rows above for it with that in mind.

Revise 360 — a product of Revise 360 Ltd, created by Olly Pettitt. An independent resource: not written, endorsed or approved by OCR or any exam board. Classroom use and personal study only.